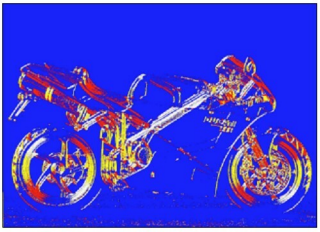

Figure 4: Angle map after quantisation

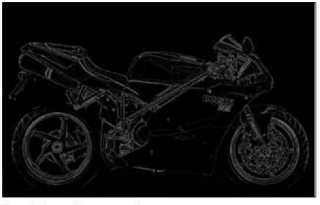

Figure 5: Non-maximum suppression

The steps are numbered in the same order as above.

1. First, the image (Figure 1) is loaded as a grayscale image array. The *ndimage* package from SciPy has routines for reading images. The *imread* function takes the filename string as input, and returns a 3D/2D array of pixel values. A Gaussian kernel with a chosen standard deviation and kernel size is applied to the image to remove any noise.
2. A Prewitt operator is applied to create two images, horizontally differentiated and vertically differentiated. Let us call them *gradx* and *grady* respectively. The gradient map is given by issuing the following command:

```
grad = squareroot(gradx^2+grady^2)
```

The angle of the gradient at each point is *arctan(grady / gradx)*.

3. Now, let's quantise the angles to help the non-maximum suppression. The division is as follows:

```
a. 0-22.5 or 157.5-202.5 or 337.5-360 => 0
b. 22.5-67.5 or 202.5-247.5       => 45 degrees
c. 67.5-112.5 or 247.5 to 292.5   => 90 degrees
d. 112.5-157.5 or 292.5-337.5     => 135 degrees.
```

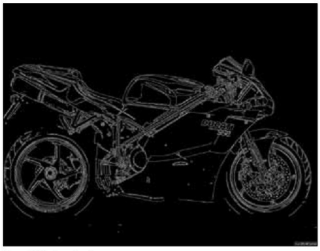

Figure 6: Hysteresis edge

Having quantised the angles, the previously discussed non-maximum suppression algorithm is run to get single-pixel-thick edges of the image.

4. The final step is hysteresis edge detection. For our test image, a *Th* of 50 and *Tl* of 5 is used. This implies that an edge is started if the pixel grayscale value is greater than 50, and is terminated when the grayscale value is less than 5. The edges are given a white colour, and the non-edges are left black. Figure 6 is the final image obtained.

The source code of the article, which can be downloaded from *http://www.linuxforu.com/article_source_code/sept12/canny_edge.zip*, is the implementation of the canny edge detection algorithm in Python. The module has a class called *Canny,* which takes in the path to the image, the sigma value for Gaussian blur, and the lower and upper thresholds. The class will finally give a binary image—0 for no edges and 255 for edges. END

References

[1] *http://bit.ly/OOQGMk*
[2] *http://en.wikipedia.org/wiki/Canny_edge_detector*
[5] You can download the source code of this article from *http://www.linuxforu.com/article_source_code/sept12/canny_edge.zip*

Acknowledgement

My humble thanks to Prof Dr R Aravind (Department of Electrical Engineering, IIT Madras) for reviewing this article and suggesting amendments.

By: Vishwanath Saragadam

The author is an undergraduate student doing his electrical engineering at IIT Madras, a coding enthusiast and freelancer animator. His code snippets can be found at *www.github.com/vishwa91/snippets* and his blog at *www.cyroforge.wordpress.com*.